

SPECIAL ISSUE PAPER

Intent-Based Network Configuration Using Large Language Models

Nguyen Tu  | Sukhyun Nam | James Won-Ki Hong 

Department of Computer Science and Engineering, POSTECH, Pohang, Korea

Correspondence: Nguyen Van Tu (tunguyen@postech.ac.kr)

Received: 19 August 2024 | **Revised:** 20 September 2024 | **Accepted:** 23 October 2024

Funding: This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (RS-2024-00392332, Development of 6G Network Integrated Intelligence Plane Technologies).

ABSTRACT

The increasing scale and complexity of network infrastructure present a huge challenge for network operators and administrators in performing network configuration and management tasks. Intent-based networking has emerged as a solution to simplify the configuration and management of networks. However, one of the most difficult tasks of intent-based networking is correctly translating high-level natural language intents into low-level network configurations. In this paper, we propose a general and effective approach to perform the network intent translation task using large language models with fine-tuning, dynamic in-context learning, and continuous learning. Fine-tuning allows a pretrained large language model to perform better on a specific task. In-context learning enables large language models to learn from the examples provided along with the actual intent. Continuous learning allows the system to improve overtime with new user intents. To demonstrate the feasibility of our approach, we present and evaluate it with two use cases: network formal specification translation and network function virtualization configuration. Our evaluation shows that with the proposed approach, we can achieve high intent translation accuracy as well as fast processing times using small large language models that can run on a single consumer-grade GPU.

1 | Introduction

Nowadays, the scale and complexity of networks are continuously increasing [1]. The increasing of traffic demand¹ has been leading to an increase in the number of devices required to accommodate this growth. Additionally, in an effort to reduce costs and increase the flexibility of network infrastructure, network functionalities are being migrated to virtual machines (VMs) or containers [2], resulting in more diverse configurations. Consequently, network administrators find it increasingly challenging to comprehend these complex networks. In such an environment, manual network management can lead to problems due to human errors and is also inefficient in swiftly responding to changes in network conditions.

For these reasons, the automation and simplification of network management have long been topics of interest. Research on

intent-based networking [3–5] (IBN) has been ongoing to enable administrators, even those without extensive specialized knowledge, to manage networks using natural language. Recently, studies have focused on leveraging artificial intelligence (AI) to automate various network configurations, and advancements in natural language processing have furthered research efforts in the IBN domain to reflect the abstract requirements of administrators.

IBN can help simplify network configuration. The term “Intent” refers to an abstract request for a desired state of the network, which can be expressed either in natural language or numerically within a predefined input format. Users or network administrators can express their desired network configuration using high-level, possibly natural language, as an intent. The intent is then translated into low-level network configuration for execution. Intent-based configuration reduces the need for writing

complex configuration files, commands, or programs, saving time and reducing potential errors. However, the intent translation step presents a significant challenge: designing a model that understands natural language intent and accurately translates it into network configuration instructions. Recent advances in large language models (LLMs) [6] provide a promising solution to this challenge.

A LLM is an AI model trained on vast amounts of text data to perform natural language processing (NLP) tasks. Typically, LLMs possess billions of parameters, enabling them to comprehend language patterns and context through extensive datasets. These models demonstrate high performance in a variety of language-based tasks, including document summarization, translation, question answering, and text generation. Currently, LLMs are emerging as revolutionary tools in various fields for language understanding and generation, and they hold significant potential for application in network automation. Recent studies have focused on exploring how these models can be practically utilized in real-world network environments, with IBN being the most active area of research leveraging LLMs. LLMs can understand human language in complex sentences and paragraphs, generating low-level instructions accordingly.

Although LLMs are good at general natural language understanding, naively applying LLMs to IBN can lead to poor accuracy. In this paper, we propose using both fine-tuning and in-context learning in LLMs to improve the intent translation performance. Fine-tuning is the process of training a pretrained model with a specific dataset tailored to a specific task, thereby allowing the model to adjust its weights to adapt to the designated task. In-context learning is a phenomenon in LLMs that allows the model to learn from examples (e.g., examples of high-level intent and expected low-level configuration instructions) provided along with the actual intent. Our contributions are as follows:

- We present a general and effective approach for intent translation in IBN using LLMs. We proposed and evaluated the usage of fine-tuning, dynamic in-context learning, and continuous learning to achieve high intent translation accuracy. The examples for in-context learning are selected from an example set. Dynamic in-context learning means that the examples are selected based on the relevance to the input intent. Continuous learning means that when the system fails to translate an intent correctly, that intent and the correct output are added to the example set for in-context learning so that future similar intents can be translated correctly.
- To show the feasibility of our approach, we implemented the system and applied it to two use cases: network function virtualization (NFV) configuration and formal specification translation. In the former use case, the system translates user intents into JSON-based configurations for virtual network functions and service function chains (SFC). In the latter use case, the system translates user intents into a JSON-based formal specifications for network routing. Details of these two use cases are presented in Section 3.4.
- We evaluated our approach in the two above use cases with various publicly available LLMs. Our evaluation showed

that using either in-context learning or fine-tuning or both can significantly improve the intent translation accuracy of small-size LLMs to be comparable with large-size LLMs. The evaluation also showed significant accuracy improvement with the continuous learning mechanism. The small-size LLMs can be deployed on consumer-grade GPUs with much smaller resource usage and faster processing time.

This paper extends our previous work [7] in several directions. First, in our previous work, we focused only on NFV configuration. In this work, we propose a more general approach that can be applied to other IBN tasks. Second, in the previous work, we used only in-context learning. In this work, we use fine-tuning and dynamic in-context learning with continuous learning. The evaluation shows that using both dynamic in-context learning and fine-tuning further improves the intent-translation accuracy. Third, in the previous work, the examples used in in-context learning were fixed. In this work, the examples for in-context learning can be chosen dynamically from the example set to be semantically close to the user intent. We also propose continuous learning where the system can learn from new intent and improves the performance overtime. Finally, we present a more thorough and detailed evaluation for both use cases with the latest publicly available LLMs at the time of writing this paper.

The rest of the manuscript is organized as follows. First, we survey the background and related work on IBN and LLMs in Section 2. Second, we present our proposed approach to applying LLMs for intent translation tasks using fine-tuning, in-context learning, and continuous learning in Section 3. Third, we show the evaluation results of our approach with two use cases in Section 4. Finally, we conclude this paper and discuss the limitations and future work in Section 5.

2 | Background and Related Work

In this section, we present the background and related work on IBN and LLMs. We provide details on fine-tuning and in-context learning in LLMs. Additionally, we cover recent studies on using LLMs in IBN systems.

2.1 | IBN

To tackle the large scale and complexity of network infrastructures, IBN has been proposed to improve and simplify network configuration, operation, and management. The term “intent” refers to an abstract request for a desired state of the network, which can be expressed either in natural language or numerically within a predefined input format. Intent describes what the network state should be but not how to achieve that state. This abstraction can help reduce complexity, thereby minimizing misconfigurations and human errors. IBN can help reduce the time and effort required from network operators and administrators, thereby lowering operation and management costs.

There are three main tasks in an IBN system: intent translation, intent implementation, and continuous assurance. Intent translation involves translating high-level expressed intents (e.g., natural language expressed intent) into low-level network

configuration instructions (e.g., JSON-based configuration files). Then, intent implementation automatically applies these low-level instructions to the network infrastructure. Finally, the IBN system needs to continuously monitor and adjust the network configuration (if needed) to ensure that the network state is aligned with the original intent.

In this work, we focus on natural language-expressed intent; i.e., network operators and administrators can express the intent freely with natural language instead of following a strict format with limited vocabulary. While intent implementation and continuous assurance can be achieved via traditional programming, translating high-level expressed intents into low-level instructions has been the most challenging task in IBN systems, as this task cannot be done effectively via the traditional programming approach. The recent breakthroughs in natural language understanding with LLMs open a new, effective approach for the intent translation task.

2.2 | LLMs

LLMs have gained significant attention recently in both industry and academia with the introduction of ChatGPT.² Typically, LLMs are built using the transformer [8] neural network architecture, which is the foundation of LLMs and has become the standard for processing text and speech data.

Recent LLMs are trained on Internet-scale datasets, including books, websites, articles, and programming source codes. LLMs possess the ability to understand and interact with humans on various aspects and concepts. Typically, LLMs range in size from several billion to hundreds of billions of parameters. This large-scale training and extensive model size enable LLMs to demonstrate high performance in a variety of language-based tasks, including document summarization, translation, question answering, and text generation. LLMs are emerging as revolutionary tools in various fields for language understanding and generation, and they hold significant potential for application in network automation.

In the field of networking, recent studies have focused on exploring how these models can be practically utilized in real-world network environments, with IBN being the most active area of research leveraging LLMs. LLMs can understand intent expressed in human language in complex sentences and paragraphs and then generate low-level instructions accordingly.

However, directly applying LLMs to intent translation or any domain-specific task does not guarantee a high level of accuracy, especially with small and medium-sized LLMs, as shown in previous work [6, 9], as well as in our evaluation. To enhance the performance of LLMs in domain-specific tasks, fine-tuning and in-context learning are commonly used methods.

2.2.1 | Fine-Tuning

While LLMs can have extensive language knowledge spread across multiple domains, their performance may not be good enough when applied to a specific domain or task. In these

cases, fine-tuning can be used. Fine-tuning is the process of taking a pretrained LLM and training it further on a smaller but task-specific dataset. By doing so, the model can be fine-tuned to adapt to that specific task and therefore performs better than the original general-purpose LLM on that specific task.

There are several techniques used for fine-tuning an LLM, each with its own advantages and disadvantages. The first method is full fine-tuning [10], which updates all the parameters of the pretrained model. Full fine-tuning allows maximizing the model's adaptation to the specific task and offers the highest possible performance. However, full fine-tuning usually requires a lot of time and computational resources.

To reduce the time and computational resources required by full fine-tuning, layer-specific fine-tuning [11] can be used, where only certain layers (e.g., later layers) of the model are fine-tuned. Layer-specific fine-tuning also reduces the chance of overfitting compared to full fine-tuning. However, the performance of layer-specific tuning is not as good as full fine-tuning.

Using adapter modules [12] is another method for resource-efficient fine-tuning of LLMs. Adapter modules are small neural networks that are added between the layers of the pretrained LLM and updated during the fine-tuning phase. The original weights of the LLM are kept fixed. Compared to full fine-tuning and layer-specific fine-tuning, adapter module fine-tuning requires less memory and computing resources.

Low-rank adaptation (LoRA) [13, 14] is currently the most resource-efficient fine-tuning method. LoRA adds low-rank matrices to the layers of the model. Only these matrices are updated during fine-tuning, and the original model weights are kept the same. Compared to full fine-tuning, layer-specific fine-tuning, and adapter modules, LoRA requires the least computing and memory resources. In this work, we use LoRA as the method for LLM fine-tuning.

2.2.2 | In-Context Learning

In-context learning is a phenomenon in LLMs where the user provides the model with several examples of input-output pairs, such as an intent and its corresponding JSON configuration. The LLM learns from these examples during inference to produce the correct output for the actual intent. Unlike training and fine-tuning, which update the model's weights, in-context learning allows the model to adapt to new tasks without altering its underlying parameters.

Researchers have explored how in-context learning works in recent studies [15, 16]. They found that LLMs may have learned many latent document-level concepts during training and then infer these during inference when there is a shared latent between examples and the actual input. They have also provided guidelines on how to better leverage in-context learning to improve accuracy and reduce the number of examples needed.

Compared to fine-tuning, in-context learning allows users to dynamically adjust the context (i.e., examples) to different

tasks and domains without retraining or redeployment of the models. In-context learning also becomes useful when the available dataset is small and not sufficient for effective fine-tuning. In this work, we use both fine-tuning and in-context learning to improve the model's performance on the intent translation task.

2.3 | IBN With LLMs

IBN has been a main research topic in many networking fields for many years. In this paper, we focus on studying the related work in the field of NFV [2] and the field of routing configuration, where we present two use cases. In the field of NFV, the authors in [17] attempt to combine NFV and IBN, enabling the automation of VNF deployment in cloud-based environments. The system supported multitenancy and intent refinement, utilizing a high-level pseudo-code for intent expression. In [18], the authors proposed an over-the-top Intent-based framework for network slicing in 5G networks. The system allowed for the automation of network slicing using declarative intents written in a specific format with a limited vocabulary. The intent was then mapped to predefined NFV blueprints, and the system used these blueprints for NFV deployment and configuration. In [19], the authors proposed a solution for intent refinement that could translate intent into a set of configurations and establish a SFC, allowing the use of a constrained set of natural language for intent expression.

In the field of routing configuration, the authors in [20] present a synthesis system for routing policy configuration in data centers. The system uses a high-level language to express the desired routing state, which is then automatically translated by a compiler into switch configurations. In [21], the authors present a system that helps network operators automatically update the access control list (ACL) in Alibaba's global WAN. The system allows the network operators to express the desired ACL using declarative language (i.e., intent). In [22], the authors leverage NLP to build an intent-driven network control system that focuses on network connectivity and rate control.

There are also other intent-based network management systems using NLP [23–25]. However, the above-mentioned work has restrictions on the high-level languages that are used to

express the intent. The restrictions are necessary when the language understanding capability of the systems is limited. With the introduction of LLMs, recent work starts to allow fully natural language-expressed intents. In [9], the authors use LLMs to translate intent into network configuration. They experiment with state-of-the-art LLMs as well as fine-tuning LLMs. They also provide guidelines for building complex tasks by dividing them into several simpler subtasks. In [26], the authors use LLMs with in-context learning to translate intents into policy configurations for application management with a demonstrated use case. In [27], authors propose a modular system for IBN using LLM with retrieval-augmented generation (RAG) [28]. However, in both [26, 27], a qualitative evaluation is not shown, such as the accuracy percentage of intent translation. In our previous work [7], we also leveraged in-context learning in LLMs to generate NFV configurations from natural language intents and evaluated the system with a hand-crafted dataset.

Compared to previous work, in this paper, we proposed to use both fine-tuning and dynamic in-context learning in LLMs for intent translation tasks. We proposed a continuous learning mechanism to enhance dynamic in-context learning. We also thoroughly evaluated our approach in two use cases with multiple state-of-the-art offline LLMs, as shown in Section 4.

3 | System Architecture and Methodology

In this section, we present the detailed system architecture and workflow of our proposed approach. We discuss the prompt design, how dynamic in-context learning with continuous learning is implemented, as well as how intent verification and the feedback loop work. Following that, we present two example use cases in detail. We also cover the implementation of the system.

3.1 | Architecture Design

Figure 1 shows the architecture of our proposed approach. The system workflow is as follows. First, the natural language intent from the user is sent to the example selection function. The example selection function searches within the database of examples (where each example is a pair of a high-level intent and its correct low-level configuration) and outputs M examples that

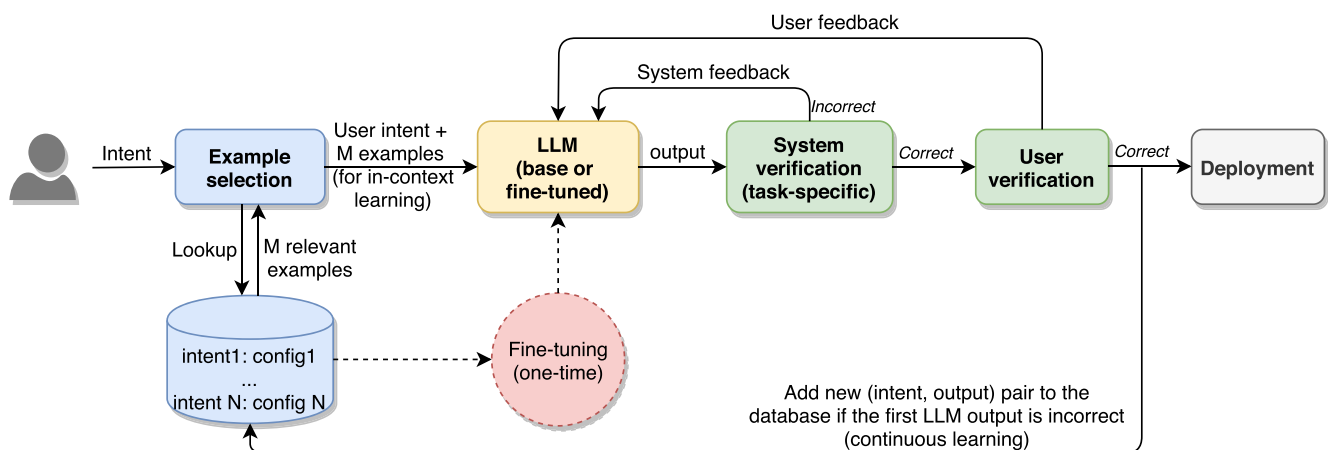


FIGURE 1 | The proposed architecture for intent translation task using LLMs.

are the most relevant to the user's intent. Next, the intent and M examples are sent to the LLM so that the intent can be translated into a low-level configuration. The LLM uses learn from the provided examples and produce output for the actual user intent (in-context learning). The output is then verified by a system verification (e.g., compiler, static checker) block to check for hard conflicts or errors such as syntax errors. The system verification block is task-specific. If the system verification fails, the system feedback, such as an error log, is sent to the LLM to generate a new configuration output until the system verification is passed. After that, the configuration output is verified by the users (e.g., network operators or administrators) to ensure that the configuration is correct. If not, human feedback is sent to the LLM by the user to generate a new output. Finally, the correct configuration is deployed into the network infrastructure. Additionally, if the first intent translation is incorrect, a pair of that intent and correct output is added to the database of examples.

The LLM can be a base model or a fine-tuned model using the same dataset that served as the examples for in-context learning. Fine-tuning is optional and only needs to be done once the pipeline is set up. Fine-tuning can greatly improve the accuracy of the intent translation task. However, the fine-tuned model is only good for the specific task it was fine-tuned for; i.e., each task needs its own fine-tuning model. Fine-tuning also requires good data preparation and a significant amount of computing resources. In this work, we use LoRA [13], which is the state-of-the-art technique for LLM fine-tuning with low memory and computing resource requirements.

3.2 | Prompt Design

The prompt is the text data fed to the LLM to get the desired output. We design the prompt to have three sections: output template, examples, and user intent. The designed output is a low-level instruction that can be read and executed by a program, so it should follow a strict template (e.g., JSON template) or a programming language (e.g., pseudo-code, Python, or Bash script). We chose a template (JSON template in particular) as the output of intent translation because it has several advantages over a programming language. First, a JSON template is already used by many configuration tools, which means that we can use them for deployment functions with little to no change. Second, LLMs are trained on Internet-scale data, which already contains many JSON examples. Third, we can limit the scope of the task easily with a template, which can help improve intent translation accuracy and reduce errors because the LLMs are constrained to produce output in the template format instead of free expression using a programming language.

3.2.1 | Dynamic Example Selection With Continuous Learning

In previous work [7, 26], the examples are fixed at the beginning. This works well when there are only a small number of examples, as all examples can be put into the prompt with the user intent. However, when there are several tens or hundreds of examples available, we cannot put all of them into the prompt due to several reasons. First, LLMs have limited context length,

i.e., the maximum number of tokens for the input. Most LLMs have a context length ranging from 2k to 8k tokens, with some LLMs extending the context length to 128k tokens [6]. Second, increasing the context size increases processing time, as shown in our evaluation. Lastly, previous work suggests that a larger context size decreases translation accuracy [9]. To automatically select relevant examples from the dataset, we choose examples that are semantically close to the user intent but also diverse and complementary to each other. This helps to improve the performance of in-context learning as shown in [29].

To calculate the semantic closeness between an intent and an example or between two examples, the text is converted into vector format using a text embedding function [10]. Text embedding converts text data into numerical vectors in a high-dimensional latent space. Words that are semantically close to each other will have shorter distances after conversion into vectors, and words that are different will have longer distances in the vector representations. The entire example set is converted to vectors using a text-embedding LLM and stored in the database. When a new intent arrives at the example selector, the selector uses the text-embedding LLM to convert the intent into a vector and then retrieves M most relevant examples from the dataset. The example selection is similar to RAG [28] with the use of text embedding and vector database, but instead of using unstructured documents, example selection uses structured example set.

The system does not only learn from the initial example set but should also be able to learn from new intents and outputs during operation, i.e., continuous learning. If the LLM produces an incorrect output for a given intent, that intent and the correct output (after human verification) will be inserted into the database as a new example. This can help the system improve accuracy over time and produce correct outputs for similar intents in the future. Since adding every new intents can grow the database quickly, we only chose useful intent to be updated, i.e., only add a new intent to the database if the first translation of that intent is incorrect. We proposed continuous learning as an extension of dynamic in-context learning and only works when dynamic in-context learning is used.

3.3 | Intent Verification and Feedback

There are several ways the intent deployment can go wrong. First, LLMs can hallucinate or produce incorrect configurations for the intent. Second, the intent itself may have conflicts such as internal conflicts within the intent or conflicts with other already deployed intents. Third, the intent may request resources that are not available or not supported by the infrastructure. The intent may also violate the capabilities of the user; for instance, the intent may ask for a real-time low-delay connection, but the current user capability does not allow that. Therefore, intent verification should be done before deployment.

We propose a two-step verification process with system verification and human verification. The output from the LLM is first sent to the system verification function to check for grammar, syntax, or task-specific issues. System verification is task-specific and can be a compiler (for grammar and syntax checking), a

resource-checking block (for checking if resources are available or if any resources or policies conflict in the infrastructure), or a combination of both. If the check fails, feedback including an error log from system verification is sent to the LLM to generate a new configuration. The loop continues until the system verification is passed or until the maximum retry limit is reached. Then, the output is inspected by the user.

In our proposed approach, human involvement is still required for the final deployment decision. The user reviews the output from the LLM and provides feedback to the LLM via a GUI using natural language (e.g., “the destination port should be 8080, not 8081”), and the LLM generates a new output. The new output goes through system verification again and is then sent to the user for another review. The user also has the option to modify the output directly for deployment without providing feedback to the LLM.

One could argue that involving humans in the loop defeats the purpose of automating the system with IBN. However, we believe that an IBN system with humans having the final decision has several advantages. First, the user needs to verify the output instead of creating it from scratch, which reduces the time and effort required, especially with high intent translation accuracy from the LLM (as shown in the evaluation). Second, the LLM can help reduce human error, and users can also check for LLM errors such as hallucinations. Therefore, the combination of both human and LLM can help minimize errors.

3.4 | Example Use Cases

We present two example use cases: NFV configuration and formal specification translation. It is important to note that our proposed approach can be applied to various other intent translation tasks. We focus on these two specific use cases because they are central to the evaluation of our approach, which will be detailed in Section 4.

3.4.1 | NFV Configuration

NFV [2] decouples network functions, such as Firewall, network address translation (NAT), deep packet inspection (DPI), intrusion detection system (IDS), and load balancer (LB), from their physical hardware, allowing them to run on standard commodity servers as virtual network functions (VNFs). NFV also allows chaining VNFs into SFCs to form different services. NFV not only reduces the cost of proprietary middleboxes but also increases the flexibility of network configuration. However, one of the challenges in NFV is the complexity and flexibility of configuration, which demands expert-level administrators.

To simplify the NFV configuration problem, we use LLMs to translate intent into NFV configuration. The implementation is integrated into the NI-testbed, or network intelligence testbed [30], an open-sourced platform for developing and experimenting with AI-based management methods for NFV. We developed NI-testbed on top of the OpenStack³ open source cloud computing infrastructure platform. The detailed architecture and implementation are presented in [7, 30].

Although NI-testbed has a closed-loop control while VNFs and SFCs are in operation, the initial configuration process is still complicated. We implemented an intent-based configuration pipeline named NFV-intent for configuring VNFs and SFCs in the testbed. Users express their intent through a prompt, such as “create an SFC with firewall and IDS. The SFC should have a maximum bandwidth of 1Gbps. The SFC should route all https traffic”. Subsequently, NFV-Intent sends this query to the LLM and retrieves the JSON configuration data for the intent. After user verification, NFV-Intent sends the JSON configuration to the deployment module in NI-testbed to set up the VNFs and SFCs accordingly. Figure 2 shows an example of intent and expected output for the NFV configuration task.

In the current prototype, we support six actions, including creating, updating, and destroying VNFs and creating, updating, and destroying SFCs. When creating a VNF, a flavor is used to indicate the specification of the VNF, including VNF type, and the amount of CPU, memory, and disk that should be used. When creating SFCs, the QoS requirement, traffic classifier, and VNF chain should be defined.

3.4.2 | Formal Specification Translation

Network configuration and routing configuration are one of the most common tasks in network management. Many network configuration tools [31, 32] allow defining a high-level formal specification of the desired configuration state. The tools can then understand and deploy that configuration into the network infrastructure. Modern network devices usually support NETCONF [33], which is a mechanism to install, modify, and delete the configuration of network devices such as switches and routers. NETCONF requires the network configuration state to be described in the form of XML or JSON-based format.

A common challenge of the above tools is that they require the users to define the desired network state in a formal language with strict rules, format, and vocabulary set, which requires a learning process. LLMs can help to translate natural language intents into formal specifications. An example of a use case and dataset of formal specification translation is presented in detail in [9]. Figure 3 shows an example of input intent and expected formal specification output. The formal specification should show the desired state of reachability, waypoints, and load-balancing policy. Reachability means the traffic from source hosts should be able to reach the destination host. Waypoints show the intermediate switches of the traffic. Load-balancing shows the different paths that the traffic should be load-balanced between.

3.5 | Implementation

To build the pipeline shown in Figure 1, we use langchain,⁴ which is an open-source framework for building LLM applications by chaining interoperable components. To run the LLM, we use ollama,⁵ which is an open-source software for deploying and managing LLMs. We use Chroma,⁶ which is an AI-native and open-source embedding database system, to store the dataset of examples. Finally, we use Streamlit⁷ to build a web interface for our system. We use unsloth⁸ for fine-tuning LLMs.

```

Input intent: "create an SFC named "hellosfc" with 3 firewalls.
              It should have a maximum bandwidth of 1000Mbps.
              Apply traffic filtering on port 80."

Expected output:
{
  "action": "create_sfc",
  "create_sfc": {
    "name": "hellosfc",
    "max_bandwidth_mbps": 1000,
    "delay_requirement": "normal",
    "flow_classifiers": [
      "0.0.0.0/0 -> 0.0.0.0/0:80-80"
    ],
    "vnfs": [
      {
        "vnf_type": "fw",
        "flavor": {
          "cpu": 1,
          "ram_gb": 2,
          "disk_gb": 10
        },
        "num_instances": 3
      }
    ]
  }
}

```

FIGURE 2 | An example of input intent and output JSON configuration for NFV configuration task. The intent is intentionally designed to sound odd to verify the language understanding capability of LLMs.

```

Input intent: "100.0.4.0/24 is accessible from zurich.
              Traffic from zurich to 100.0.4.0/24 passes through basel.
              Traffic originating from warsaw can reach the subnet 100.0.9.0/24.
              The route from warsaw to 100.0.9.0/24 includes rotterdam."

Expected output:
{
  "reachability": {
    "zurich": [
      "100.0.4.0/24"
    ],
    "warsaw": [
      "100.0.9.0/24"
    ]
  },
  "waypoint": {
    "(zurich,100.0.4.0/24)": [
      "basel"
    ],
    "(warsaw,100.0.9.0/24)": [
      "rotterdam"
    ]
  },
  "loadbalancing": {}
}

```

FIGURE 3 | An example of input intent and output JSON configuration for formal specification translation task from the dataset [9].

For the NFV configuration use case, besides the evaluation, we also integrated the intent pipeline into NI-testbed [30] and verified the operation. For the formal specification translation use case, we only evaluated the system with the public dataset [9].

4 | Evaluation

In this section, we evaluate two aspects: the accuracy of intent translation and the processing time from when the input intent is sent to when the complete output JSON configuration is received. We conducted the evaluation using a total of six LLMs and one fine-tuned LLM, as shown in Table 1. The models were

executed on a machine equipped with two Intel Xeon Silver 4216 CPUs with hyper-threading enabled, 128GB of RAM, and an Nvidia Quadro RTX A6000 GPU with 48GB of VRAM.

For the NFV configuration evaluation, we used a semi-automatic method to create the test set: We utilized Microsoft Copilot⁹ to read the NFV configuration template and then asked Microsoft Copilot to generate 40 different intents and JSON configuration pairs based on the template. The intents were spread over six actions: “create_vnf,” “create_sfc,” “update_vnf,” “update_sfc,” “destroy_vnf,” and “destroy_sfc.” Then, we manually verified that the intent and JSON configuration were correct. However, the same intent can be expressed in many ways using

natural language. To verify the accuracy of translating different expressions of the same intent, we used Microsoft Copilot to create three variations of each intent (i.e., change words, word orders, and the way of intent expression). In total, we have 120 pairs of intent and JSON configuration in the test set. For the example set, we manually created nine intent-output pairs that spread all over the fields in the template. Since the example set is small, we did not fine-tune any model on this use case. While we could create a bigger dataset, it would be very time-consuming. Instead, we take this as an opportunity to demonstrate the use case where big dataset is not available.

For the formal specification translation use case, we used the NetConfEval dataset [9] with a 0.5:0.5 split ratio for training

TABLE 1 | List of offline LLMs used in evaluation.

Model	Description
llama3.1:8b, llama3.1:70b ^a	Open source LLM, 8B and 70B parameters version released by Meta
gemma2:9b, gemma2:27b ^b	Open source LLM, 9B and 27B parameters version released by Google
qwen2:7b, qwen2:72b ^c	Open source LLM, 7B and 72B parameters version released by Alibaba
formal_spec_ft	Our fine-tuned model for formal specification translation task, based on llama3.1:8b

^allama3.1, <https://llama.meta.com>.

^bgemma2, <https://huggingface.co/google>.

^cqwen2, <https://github.com/QwenLM/Qwen2>.

and testing for the formal specification translation evaluation. Compared to NFV configuration dataset, the NetConfEval dataset is bigger with over 1500 data points; however, the dataset was procedurally generated, and therefore, the variation of natural language expression could be limited. Note that in NetConfEval [9], the author considers a prompt with multiple sentences, each for different policies, as a batch of different intents. For example, the input in Figure 3 is a batch of four intents. The outputs of these intents are merged into one JSON output. We argue that such a prompt should be considered as one complex intent with a complexity index of 4, i.e., a combination of four base intents. Furthermore, in batch processing, the outputs should not be merged. Nevertheless, this does not affect the usefulness of the dataset and the purpose of our evaluation. While the NetConfEval dataset has several intents with high complexity, we only use a subset of the dataset with a complexity index less than 27 (i.e., a combination of less than 27 base intents) because the high complexity intents in the dataset frequently exceed LLM context size [9].

The test set is used for evaluation, while the train set is used for both fine-tuning and example selection. formal_spec_ft is fine-tuned based on llama3.1:8b LLM with 4-bit quantization using unsloth framework. We ran the fine-tuning process with a learning rate of 0.0002 in three epochs. While we chose llama3.1:8b as the base model for fine-tuning, fine-tuning should bring similar performance gain if applied to other base models.

4.1 | Performance Comparison

We evaluated the intent translation accuracy and average processing time of all models for the NFV configuration use case (Table 2) and the formal specification translation use case (Table 3). Dynamic in-context learning and continuous learning

TABLE 2 | Accuracy and processing time of six LLMs on the test set in the NFV configuration use case.

Model	qwen2:7b	llama3.1:8b	gemma2:9b	gemma2:27b	qwen2:72b	llama3.1:70b
Accuracy (%)	99.17%	97.41%	99.17%	98.33%	93.33%	90.0%
Base Accuracy (%)	57.5%	60.0%	30.0%	62.5%	51.667%	72.5%
Avg processing time (s)	1.3	1.4	2.0	3.5	5.6	5.0
Base avg processing time (s)	1.2	1.2	2.0	3.6	4.9	5.9

TABLE 3 | Accuracy and processing time of seven LLMs on the test set in the formal specification translation use case.

Model	formal_spec_ft	qwen2:7b	llama3.1:8b	gemma2:9b	gemma2:27b	qwen2:72b	llama3.1:70b
Accuracy (%)	98.81%	95.05%	94.9%	97.41%	98.26%	99.08%	99.63%
Base accuracy (%)	80.49%	86.74%	80.49%	59.58%	98.01%	99.28%	99.33%
Avg processing time (s)	2.4	2.6	2.4	3.9	7.1	9.9	8.7
Base avg processing time (s)	1.5	1.6	1.7	3.2	5.9	8.5	8.3

are enabled. The number of examples selected for in-context learning is 1 for qwen2:72b and llama3.1:70b and is 9 for other models. We did not evaluate qwen2:72b and llama3.1:70b for a higher number of examples for in-context learning due to stability and timeout issues when the size of the input prompt is large. We compared them with the base performance values where no example was provided for in-context learning, i.e., in-context learning and continuous learning were not used.

Generally, dynamic in-context learning with continuous learning significantly improves the intent translation accuracy, especially when the base accuracy is low. In the case of NFV configuration, both qwen2:7b and gemma2:9b achieved the highest accuracy at 99.17%, with gemma2:27b and llama3.1:8b achieving 98.33% and 97.41% accuracy, respectively. Surprisingly, although qwen2:72b and llama3.1:70b are significantly bigger models compared to the others, they did not perform as well compared to other models. The reason could be that the number of examples was not enough. However, when calculating the improvement over the base case with no example for in-context learning, the accuracy improvement is significant for all models. The accuracy improvement was the highest for Gemma2:9b at a 69.17% increase. While the accuracy improvement in the llama3.1:70b was the lowest at 27.5%, it was still a huge improvement.

In the case of formal specification translation, the accuracy improvement was significant in the case of formal_spec_ft, qwen2:7b, llama3.1:8b, and gemma2:9b. However, there was no accuracy improvement for the bigger models: gemma2:27b, qwen2:72b, and llama3.1:70b. For these three biggest models, the base accuracy percentages were already quite high. For the other smaller models, formal_spec_ft has the highest accuracy at 98.81% with an 18.32% accuracy improvement over the base, which is llama3.1:8b. Similar to the NFV configuration evaluation, Gemma2:9b was also the model with the highest accuracy improvement at 37.83%.

In terms of processing time, using in-context learning did not increase the processing time significantly in the case of NFV configuration use cases. In the case of formal specification translation, the processing time was increased significantly in terms of percentage. However, the absolute values were small enough for the purpose of intent translation, especially with

small models. The average processing times of three small models were less than 2 s in the case of NFV configuration and less than 4 s in the case of formal specification translation.

Figure 4 illustrates the detail cumulative distribution function (CDF) of processing time for the best accuracy small models and best accuracy big models for both use cases. In the case of NFV configuration, 99th processing time of qwen2:7b and qwen2:72b were 3.0 and 15.1 s, respectively. In the case of formal specification, 99th processing time of formal_spec_ft and llama3.1:70b were 9.2 and 38.5 s, respectively. Intuitively, the processing time increased quickly with the model size in both cases. In the case of big models, the higher processing time can lead to an unsatisfying experience for the users. More importantly, big models require more GPU memory and computing resources, which increases the infrastructure cost.

In overall, we found that with our approach, small LLMs can be used for intent translation with high accuracy comparable to or even exceeding big models while still achieving fast processing time and low memory and computing resource requirements. In the case of NFV configuration, applying the proposed approach to qwen2:7b achieved the best combination of accuracy (99.17%) and processing time (1.3 s). In the case of formal specification, applying the proposed approach to llama3.1:8b (formal_spec_ft was fine-tuned based on llama3.1:8b) achieved the best combination of accuracy (98.81%) and processing time (2.4 s).

4.2 | Ablation Study

In this section, we evaluated the contribution of fine-tuning, dynamic in-context learning, and continuous learning to the accuracy. In terms of processing time, only in-context learning contributed to the increase in the processing time by increasing the prompt size. Continuous learning happened outside of the main processing and only when the LLM output is incorrect; therefore, it did not contribute to the processing time.

Figure 5 showed the ablation results for the NFV configuration use case. In all models, both in-context learning and continuous learning contributed significantly to the performance gain. For the optimal case with qwen2:7b, in-context learning

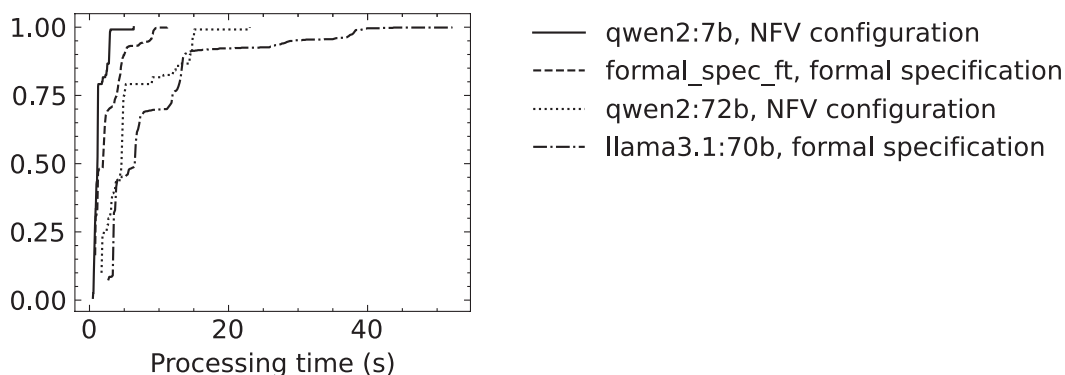


FIGURE 4 | Cumulative distribution function (CDF) of processing time of best small model and best big model for NFV configuration and formal specification translation.

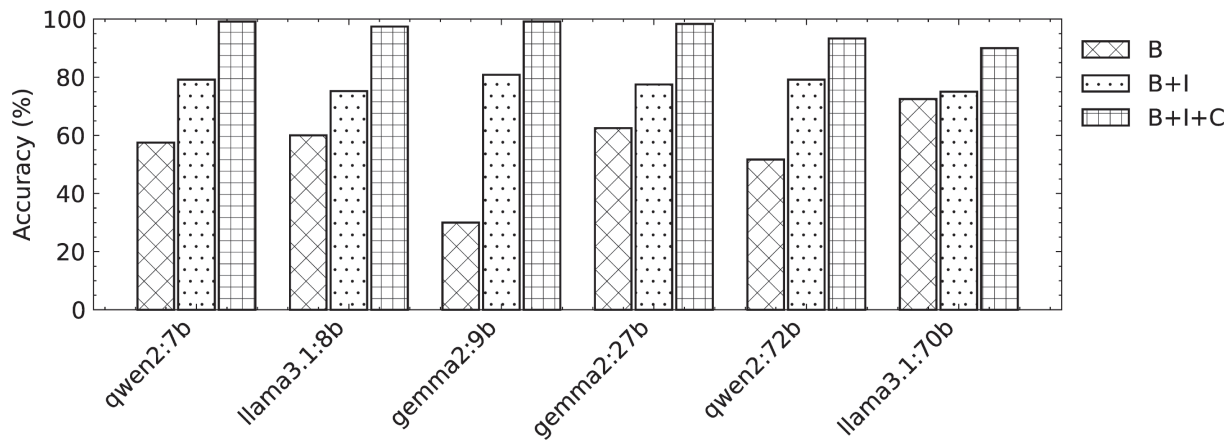


FIGURE 5 | Ablation results for NFW configuration use case. B: base model. B+I: base model with in-context learning. B+I+C: base model with in-context learning and continuous learning.

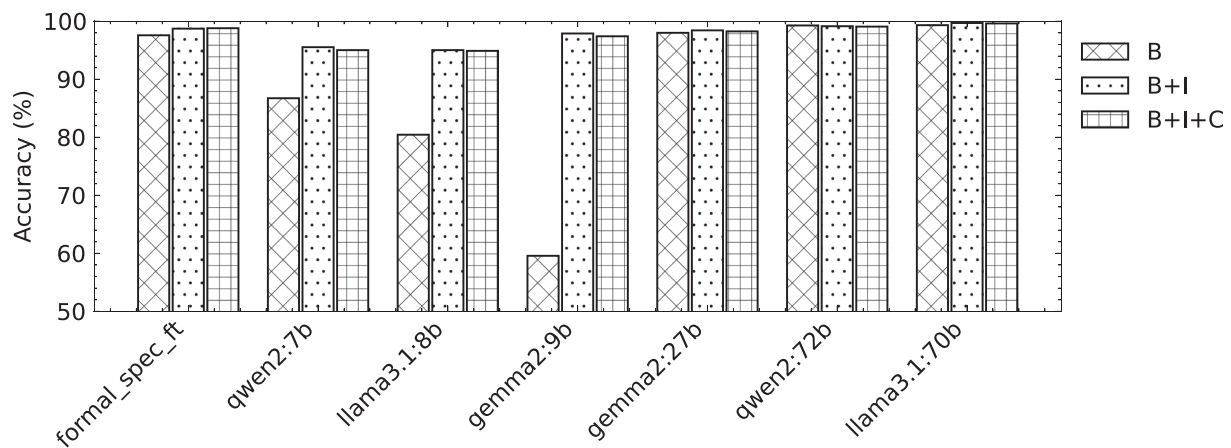


FIGURE 6 | Ablation results for NFW formal specification translation use case. B: base model. B+I: base model with in-context learning. B+I+C: base model with in-context learning and continuous learning.

improved the accuracy by 21.66%, and continuous learning added 20% more accuracy on top of in-context learning. The case of gemma2:9b is where we saw the highest improvement, where in-context learning increased the accuracy by 60.83% and continuous learning increased an additional 18.33% accuracy on top of that. One could argue that the NFW configuration dataset had intent variation (i.e., there are intent with same meaning but different way of expression) which could benefit the continuous learning mechanism. However, this is the purpose of continuous learning: to learn from an intent so that it can translate future similar intents correctly.

Figure 6 showed the ablation results for the formal specification use case. We see significant improvement from in-context learning in the case of gemma2:9b, llama3.1:8b, and qwen2:7b with accuracy improvements of 39.32%, 14.58%, and 8.8%, respectively. In the case of formal_spec_ft, fine-tuning increased the accuracy by 17.14% on top of llama3.1:8b, and in-context learning improved the accuracy by 1.14% on top of the fine-tuned model, which was a good improvement considering the model already achieved very high accuracy (97.59%) without in-context learning. We did not see any improvement from using continuous learning in this test, which was different from the

NFW configuration use case. We believe that the differences came from the example set. The NFW configuration example set was small, and the template is complicated; therefore, learning from new intents is important. The formal specification dataset is huge, and the intents are procedurally generated; therefore, it was more likely that for a user intent, there were similar intents in the example set.

4.3 | Effect of Number of Examples

In this test, we evaluated the effect when changing the number of examples selected for in-context learning, i.e., how it affected the accuracy and processing time.

Figures 7 and 8 showed the change in intent translation accuracy for NFW configuration and formal specification translation when the number of examples for in-context learning changed between 0, 1, 3, 6, and 9, except for qwen2:72b and llama3.1:70b. Overall, the first example contributed the most to the accuracy improvement. With qwen2:7b in the NFW configuration use case, the first example contributed a 31.67% performance gain, while subsequent increases to 3, 6, and

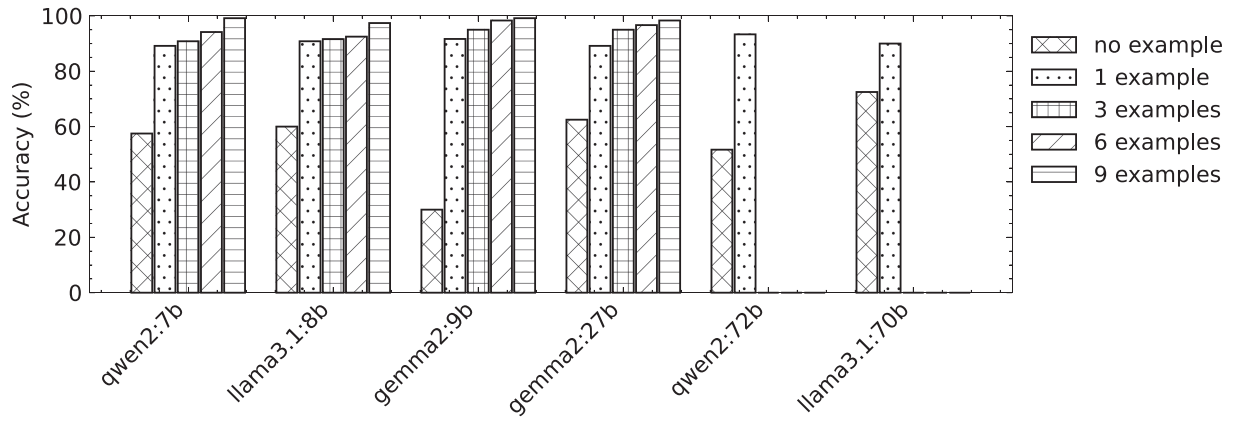


FIGURE 7 | Effect of the number of examples for in-context learning to the intent translation accuracy in the NfV configuration use case.

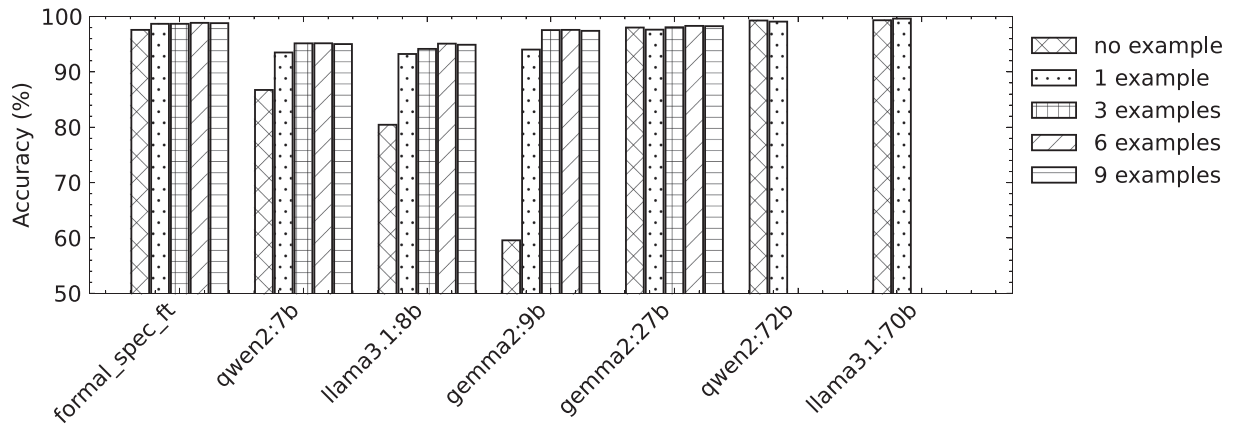


FIGURE 8 | Effect of the number of examples for in-context learning to the intent translation accuracy in the formal specification translation use case.

9 examples contributed 1.67%, 3.33%, and 5% performance gains, respectively. However, these latter performance gains were significant considering that they were improvements on top of an already high accuracy number. In the case of formal specification translation, gemma2:9b, llama3.1:8b, and qwen2:7b were the three cases that had significant improvement with the first example.

Figures 9 and 10 showed the average processing time for NfV configuration and formal specification use cases when the number of examples for in-context learning changed between 0, 1, 3, 6, and 9. As the number of examples increased, we expected the processing time to also increase as the input prompt size increased. This is generally true in the case of formal specification translation. However, in the case of NfV configuration, the processing time did not change significantly when the number of examples increased. Interestingly, with gemma2:9b, gemma2:27b, and llama3.1:70b, the processing time decreased when the number of examples increased from 0 to 1. We also observed a similar phenomenon in the formal specification use case with gemma2:9b and gemma2:27b. After looking at the detailed log, we found that the above phenomenon can happen when the translation accuracy is low and the LLM produces not only wrong but also long output text, and the longer the output, the longer the processing time. When the accuracy increased, these cases were reduced, resulting in an improvement in average processing time.

4.4 | Performance With ChatGPT

Although we focused on offline LLMs in this work, we also ran an experiment on the NfV configuration use case with nine examples. ChatGPT was able to achieve 100% translation accuracy. Previous work [9] also shows 100% translation accuracy on the formal specification translation dataset. It is intuitive because ChatGPT is considered the best-performing LLM and probably has a much larger number of parameters compared to the offline LLMs that we used in this work (at the time of writing, the number of parameters of ChatGPT is not disclosed, though).

However, compared to ChatGPT, running offline LLMs has several advantages. First, running offline LLMs can be both cheaper and faster. Second, offline LLMs can be easily fine-tuned. Third, running offline LLMs can ensure security and privacy because all the data and requests are stored internally within the company or organization.

4.5 | Evaluation Summary

The evaluation results suggested some guidelines on how to use LLMs for intent translation purposes. Fine-tuning can be useful when we have a large dataset (i.e., hundreds of input-output pairs for training). Dynamic example selection for in-context

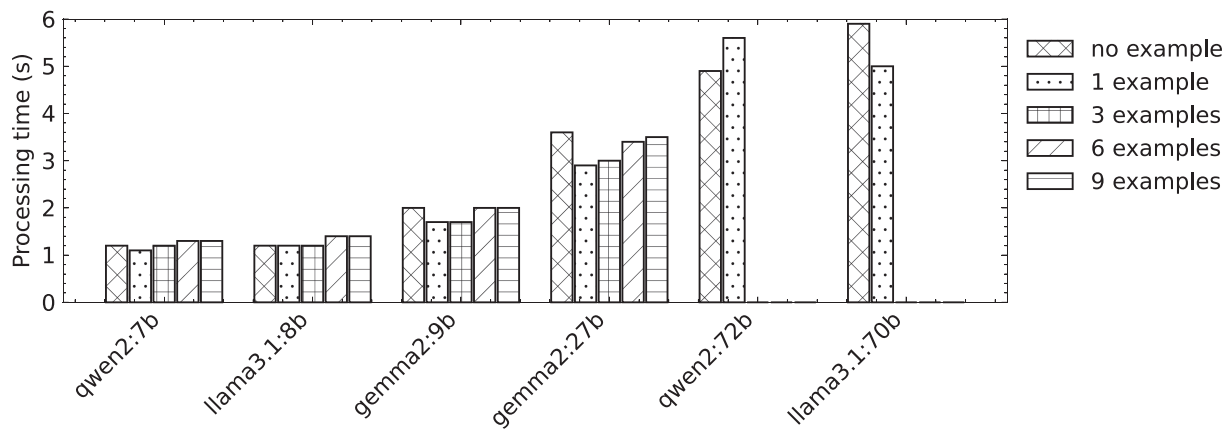


FIGURE 9 | Effect of the number of examples for in-context learning to the processing time in the NFW configuration use case.

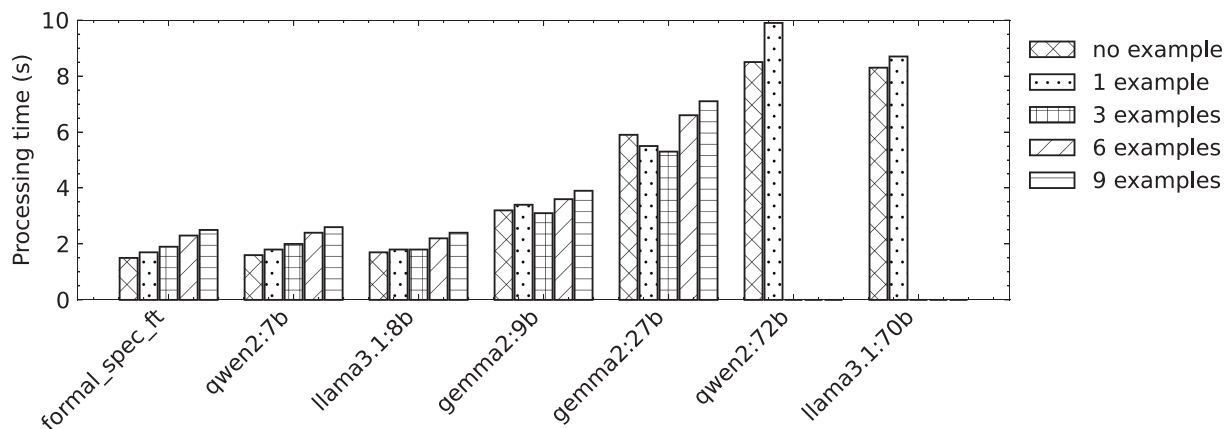


FIGURE 10 | Effect of the number of examples for in-context learning to the processing time in the formal specification translation use case.

learning is useful regardless of dataset size. Continuous learning can be useful, especially when you have few examples in the dataset. A proper evaluation with different base LLMs and a carefully designed dataset is important.

5 | Conclusion Remarks

In this section, we summarize the problem and our contribution presented in this paper. We also discuss the current limitations and the plan for future work.

5.1 | Conclusion

As the complexity and flexibility of network services and infrastructures continue to increase, there is an urgent need for a simplified, more efficient, and error-resistant network management solution. IBN can help network operators and administrators manage complex networks through high-level natural language intents. However, translating these intents into low-level network configurations remains a key challenge.

In this paper, we proposed a system to translate intents into network configurations using LLMs, leveraging three

mechanisms: fine-tuning, dynamic in-context learning, and continuous learning. We presented and discussed the system design in detail. We evaluated our proposal with two use cases: NFW configuration and formal specification translation. The evaluation results showed that our approach can improve translation accuracy significantly compared to the base LLMs. Additionally, with the continuous learning mechanism, the system can also learn from failed translations and improve accuracy over time. As such, this work contributes to the ongoing efforts of simplifying and automating network management, enabling fast and reliable network operations and management in the future.

5.2 | Future Work

Intent verification before deployment is critical; therefore, we proposed two-stage verification: system verification and user verification. We strive to reduce the need for user feedback as much as possible. There are additional verification methods that can be used to further reduce the need for user manual modification. One method is to use another LLM, such as CriticGPT,¹⁰ which is built to find the incorrect output of ChatGPT and LLMs in general. Another method is to deploy the intent into a digital twin system [34] to verify the intent before deploying to production. However, developing CriticGPT-like LLMs or a

digital twin system is also a big challenge. For future work, we plan to build such an LLM for output verification.

We have not evaluated the scalability of the system in this paper. In a large system, there may be many intents that need to be translated at the same time. We plan to evaluate the throughput of the LLMs in future work. Note that in the current test, we did not do batch processing of the intents. The throughput of LLMs can be greatly improved if batch processing is used. We also have not yet conducted a full evaluation on verification. A full evaluation of system verification requires system-specific implementation and deployment for each use case, and a full evaluation of human verification requires actual deployment and operation. This is a huge challenge that we plan to address in the future work.

In our future work, we plan to use an LLM to check the feasibility of the intent before the example selection step, i.e., whether the intent is completely out-of-scope or too ambiguous for translation. In the current approach, although these issues are addressed through intent verification and feedback steps, they result in unnecessary use of computing resources. Early checking can be done by the same LLM used for intent translation.

In this work, we considered two use cases where the scope of the output is limited to JSON templates. This is usually enough for network configuration. We plan to expand our approach to new use cases such as software and framework installation and configuration, where the scope is bigger. Kubernetes¹¹ configuration is an popular use case. How to design a dataset for such use cases remains a challenge.

Acknowledgments

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (RS-2024-00392332, Development of 6G Network Integrated Intelligence Plane Technologies).

Data Availability Statement

The data and source code that support the findings of this study are available from the corresponding author, Nguyen Van Tu, upon reasonable request.

Endnotes

- ¹ Internet Traffic report, <https://www.itu.int/itu-d/reports/statistics/2023/10/10/ff23-internet-traffic>.
- ² chatgpt, <https://platform.openai.com/docs/models>.
- ³ openstack, <https://www.openstack.org>.
- ⁴ langchain, <https://github.com/langchain-ai/langchain>.
- ⁵ ollama, <https://github.com/ollama/ollama>.
- ⁶ chroma, <https://github.com/chroma-core/chroma>.
- ⁷ streamlit, <https://github.com/streamlit/streamlit>.
- ⁸ unsloth, <https://github.com/unslothai/unsloth>.
- ⁹ Microsoft Copilot, <https://www.bing.com/chat>.
- ¹⁰ CriticGPT, <https://criticgpt.org>.
- ¹¹ Kubernetes, <https://kubernetes.io>.

References

1. A. Dogra, R. K. Jha, and S. Jain, "A Survey on Beyond 5G Network With the Advent of 6G: Architecture and Emerging Technologies," *IEEE Access* 9 (2021): 67512–67547.
2. R. Mijumbi, J. Serrat, J. L. Gorricho, N. Bouten, F. D. Turck, and R. Boutaba, "Network Function Virtualization: State-of-the-Art and Research Challenges," *IEEE Communications Surveys Tutorials* 18 (2016): 236–262.
3. A. Leivadeas and M. Falkner, "A Survey on Intent-Based Networking," *IEEE Communications Surveys & Tutorials* 25, no. 1 (2023): 625–655.
4. B. K. Saha, D. Tandur, L. Haab, and L. Podleski, "Intent-Based Networks: An Industrial Perspective," in *Proceedings of the 1st International Workshop on Future Industrial Communication Networks*, FICN '18 (New York, NY, USA: Association for Computing Machinery, 2018), 35–40.
5. A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura, "Intent-Based Networking - Concepts and Definitions," 9315, Request for Comments, RFC Editor, (2022), <https://www.rfc-editor.org/info/rfc9315>.
6. W. X. Zhao, K. Zhou, J. Li, et al., "A Survey of Large Language Models," (2023), arXiv preprint arXiv:2303.18223.
7. N. Van Tu, J.-H. Yoo, and J. W.-K. Hong, "Towards intent-Based Configuration for Network Function Virtualization Using In-Context Learning in Large Language Models," in *Noms 2024-2024 IEEE Network Operations and Management Symposium* (2024), 1–8.
8. A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention Is All You Need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17 (Red Hook, NY: Curran Associates Inc., 2017), 6000–6010.
9. C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?," *Proceedings Of The ACM On Networking* 2, no. CoNEXT2 (2024): 1–25.
10. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding," (2018), CoRR, <http://arxiv.org/abs/1810.04805>.
11. J. Howard and S. Ruder, "Fine-Tuned Language Models for Text Classification," (2018), CoRR, <http://arxiv.org/abs/1801.06146>.
12. N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., "Parameter-Efficient Transfer Learning for NLP," (2019), CoRR, <http://arxiv.org/abs/1902.00751>.
13. E. J. Hu, Y. Shen, P. Wallis, et al., "LoRa: Low-Rank Adaptation of Large Language Models," (2021), CoRR, <https://arxiv.org/abs/2106.09685>.
14. T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLORA: Efficient Finetuning of Quantized LLMs," in *Advances in Neural Information Processing Systems*, eds. A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Vol. 36 (Curran Associates, Inc., 2023), 10088–10115, https://proceedings.neurips.cc/paper_files/paper/2023/file/1feb87871436031bdc0f2beaa62a049b-Paper-Conference.pdf.
15. S. M. Xie, A. Raghunathan, P. Liang, and T. Ma, "An Explanation of In-Context Learning as Implicit Bayesian Inference," (2021), arXiv preprint arXiv:2111.02080.
16. S. Min, X. Lyu, A. Holtzman, et al., "Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?," (2022), arXiv preprint arXiv:2202.12837.
17. A. Leivadeas and M. Falkner, "VNF Placement Problem: A Multi-Tenant Intent-Based Networking Approach," in *2021 24th Conference on Innovation in Clouds, Internet and Networks and Workshops (ICIN)* (2021), 143–150.
18. F. Aklamanu, S. Randriamasy, and E. Renault, "Demo: Intent-Based 5G IoT Application Network Slice Deployment," in *2019 10th International Conference on Networks of the Future (NOF)* (2019), 141–143.

19. E. J. Scheid, C. C. Machado, M. F. Franco, et al., "Inspire: Integrated NFV-Based Intent Refinement Environment," in *2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)* (2017), 186–194.
20. S. Ramanathan, Y. Zhang, M. Gawish, et al., "Practical Intent-Driven Routing Configuration Synthesis," in *20th Usenix Symposium on Networked Systems Design and Implementation (NSDI 23)* (Boston, MA: USENIX Association, 2023), 629–644, <https://www.usenix.org/conference/nsdi23/presentation/ramanathan>.
21. B. Tian, X. Zhang, E. Zhai, et al., "Safely and Automatically Updating In-Network ACL Configurations With Intent Language," in *Proceedings of the ACM Special Interest Group on Data Communication, SIGCOMM '19* (New York, NY, USA: Association for Computing Machinery, 2019), 214–226.
22. A. Alsudais and E. Keller, "Hey Network, Can You Understand Me?," in *2017 IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)* (2017), 193–198.
23. H. Mahtout, M. Kiran, A. Mercian, and B. Mohammed, "Using Machine Learning for Intent-Based Provisioning in High-Speed Science Networks," in *Proceedings of the 3rd International Workshop on Systems and Network Telemetry and Analytics, SNTA '20* (New York, NY, USA: Association for Computing Machinery, 2020), 27–30.
24. E. J. Scheid, P. Widmer, B. B. Rodrigues, M. F. Franco, and B. Stiller, "A Controlled Natural Language to Support Intent-Based Blockchain Selection," in *2020 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)* (2020), 1–9.
25. A. S. Jacobs, R. J. Pfitscher, R. H. Ribeiro, et al., "Hey, Lumi! Using Natural Language for Intent-Based Network Management," in *2021 Usenix Annual Technical Conference (Usenix ATC 21)* (USENIX Association, 2021), 625–639, <https://www.usenix.org/conference/atc21/presentation/jacobs>.
26. K. Dzevaroska, J. Lin, A. Tizghadam, and A. Leon-Garcia, "LLM-Based Policy Generation for Intent-Based Management of Applications," in *2023 19th International Conference on Network and Service Management (CNSM)* (2023), 1–7.
27. A. Fuad, A. H. Ahmed, M. A. Riegler, and T. Čičič, "An Intent-Based Networks Framework Based on Large Language Models," in *2024 IEEE 10th International Conference on Network Softwarization (NetSoft)* (2024), 7–12.
28. P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS '20* (Red Hook, NY, USA: Curran Associates Inc., 2020), 9459–9474.
29. X. Ye, S. Iyer, A. Celikyilmaz, V. Stoyanov, G. Durrett, and R. Pasunuru, "Complementary Explanations for Effective In-Context Learning," in *Findings of ACL* (2023), 4469–4484.
30. S. Lange, N. V. Tu, S.-Y. Jeong, et al., "A Network Intelligence Architecture for Efficient VNF Lifecycle Management," *IEEE Transactions on Network and Service Management* 18, no. 2 (2021): 1476–1490.
31. A. El-Hassany, P. Tsankov, L. Vanbever, and M. Vechev, "Network-Wide Configuration Synthesis," in *Computer Aided Verification*, eds. R. Majumdar and V. Kunčak (Cham: Springer International Publishing, 2017), 261–281.
32. A. El-Hassany, P. Tsankov, L. Vanbever, and M. Vechev, "NetComplete: Practical Network-Wide Configuration Synthesis With Autocompletion," in *15th Usenix Symposium on Networked Systems Design and Implementation (NSDI 18)* (Renton, WA: USENIX Association 2018), 579–594, <https://www.usenix.org/conference/nsdi18/presentation/el-hassany>.
33. R. Enns, M. Björklund, A. Bierman, and J. Schönwälder, "Network Configuration Protocol (NETCONF)," 6241, Request for Comments, RFC Editor, (2011), <https://www.rfc-editor.org/info/rfc6241>.
34. B. R. Barricelli, E. Casiraghi, and D. Fogli, "A Survey on Digital Twin: Definitions, Characteristics, Applications, and Design Implications," *IEEE Access* 7 (2019): 167653–167671.